

Universidad Europea

PRÁCTICA 4 BACKEND EN PYTHON

API Elección Libre



Autor:
Álvaro Hidalgo Rodríguez

4 Enero 2026

Contents

1	Introducción	3
2	Arquitectura de la aplicación	3
2.1	Proyecto just_globo	3
2.2	Aplicación api_view_login	3
2.3	Aplicación api_view_roles	4
2.4	Aplicación viewset_usuarios	4
2.5	Aplicación api_view_establecimiento	4
2.6	Aplicación viewset_productos	4
2.7	Aplicación api_view_pedidos	4
3	Aplicación api_view_roles (Roles)	4
3.1	Modelo Role	4
3.2	Serialización	5
3.3	Vista y rutas	5
3.4	Autenticación y permisos	5
3.5	Operaciones disponibles y respuestas	5
4	Aplicación viewset_usuarios (Usuarios)	6
4.1	Modelo User	6
4.2	Serialización	7
4.3	Vista, rutas y autenticación	7
4.4	Operaciones disponibles y respuestas	8
4.5	Tipos de respuesta (forma del payload)	8
5	Aplicación api_view_login (Login y autenticación)	9
5.1	Rutas expuestas	9
5.2	Función login_view	9
5.3	Cookie de autenticación	10
5.4	Clase CookieJWTAuthentication	10
5.5	Respuestas posibles del endpoint POST /auth/login/	10
6	Aplicación api_view_establecimiento (Establecimientos)	11
6.1	Modelo Establecimiento	11
6.2	Rutas expuestas	11
6.3	Autenticación y permisos	11
6.4	Análisis de las funciones	12
6.4.1	list_establecimientos	12
6.4.2	create_establecimiento	12
6.4.3	detail_establecimiento	12
6.4.4	update_establecimiento	12
6.4.5	delete_establecimiento	13
6.5	Tipos de respuesta	13

7	Aplicación api_view_pedidos (Pedidos)	13
7.1	Modelo Pedido	13
7.2	Rutas expuestas	14
7.3	Autenticación y permisos	14
7.4	Análisis de las funciones	14
7.4.1	create_pedido	14
7.4.2	list_pedidos	14
7.4.3	detail_pedido	15
7.4.4	update_pedido	15
7.4.5	delete_pedido	15
7.5	Tipos de respuesta	15
8	Aplicación api_view_pedidos (Pedidos)	16
8.1	Modelo Pedido	16
8.2	Rutas expuestas	16
8.3	Autenticación y control de acceso	16
8.4	Análisis de las funciones	17
8.4.1	create_pedido	17
8.4.2	list_pedidos	17
8.4.3	detail_pedido	17
8.4.4	update_pedido	17
8.4.5	delete_pedido	18
8.5	Tipos de respuesta	18
9	Cómo ejecutar la aplicación	18
9.1	1) Clonar el repositorio	18
9.2	2) Crear (opcional) y activar entorno virtual	19
9.3	3) Instalar dependencias	19
9.4	4) Migraciones y arranque del servidor	19

1 Introduccion

Este proyecto documenta el diseño y desarrollo de una aplicación de reparto de pedidos inspirada en plataformas de entrega bajo demanda como Glovo. La aplicación tiene como objetivo modelar de forma coherente el funcionamiento básico de un sistema de delivery moderno, abordando tanto la gestión de los distintos actores implicados como los procesos fundamentales asociados a la realización y seguimiento de pedidos.

En el sistema se gestionan entidades clave como usuarios, establecimientos, productos y pedidos, así como los mecanismos necesarios para la autenticación y control de acceso. Estas funcionalidades permiten representar de manera estructurada las interacciones habituales entre clientes y comercios, ofreciendo una visión global del flujo operativo de la aplicación desde la selección de productos hasta la formalización del pedido.

La documentación presentada no solo describe los elementos funcionales del sistema, sino que sirve como punto de partida para comprender su arquitectura, sus decisiones de diseño y su posible evolución. A partir de esta base, el lector puede profundizar en los distintos componentes de la aplicación y analizar cómo se integran para dar soporte a un servicio de reparto completo y coherente.

2 Arquitectura de la aplicación

La aplicación se divide en varias aplicaciones que cada una gestiona una funcionalidad de la aplicación. Las diferentes aplicaciones de las que se componen son: Usuarios, Productos, Roles, Login, Pedidos y Establecimientos.

2.1 Proyecto just_globo

El proyecto raíz contiene la configuración general de la aplicación, incluyendo los parámetros de conexión a la base de datos, la integración de Django REST Framework y la gestión de la autenticación mediante tokens JWT almacenados en cookies HTTP-only. Asimismo, define el archivo de enrutado principal, desde el cual se exponen los endpoints de cada una de las miniaplicaciones.

2.2 Aplicación api_view_login

Esta aplicación se encarga de la autenticación de los usuarios. Implementa el endpoint de inicio de sesión, encargado de validar las credenciales del usuario y generar los tokens JWT correspondientes. El token de acceso se almacena en una cookie segura, que es utilizada posteriormente para autenticar las peticiones realizadas al resto de la API.

2.3 Aplicación `api_view_roles`

Gestiona los roles del sistema, permitiendo definir distintos perfiles de usuario con capacidades diferenciadas. Incluye la lógica necesaria para crear, consultar, modificar y eliminar roles, así como los mecanismos de control de acceso que restringen determinadas operaciones a usuarios con privilegios administrativos.

2.4 Aplicación `viewset_usuarios`

Esta miniaplicación gestiona los usuarios de la plataforma. Define el modelo de usuario, incluyendo información personal, credenciales y el rol asociado. Proporciona endpoints para la administración de usuarios y aplica restricciones de acceso en función del rol del usuario autenticado.

2.5 Aplicación `api_view_establecimiento`

Se encarga de la gestión de los establecimientos registrados en la plataforma. Modela la información básica de cada comercio y su relación con el usuario propietario. Incluye endpoints para la creación, consulta, actualización y eliminación de establecimientos, aplicando validaciones de permisos según el rol del usuario y su relación con el establecimiento.

2.6 Aplicación `viewset_productos`

Esta aplicación gestiona el catálogo de productos disponibles. Define los productos ofrecidos por cada establecimiento, incluyendo su información descriptiva y su precio. Permite realizar operaciones de gestión sobre los productos, restringiendo las acciones de modificación a usuarios autorizados.

2.7 Aplicación `api_view_pedidos`

Implementa la lógica asociada a los pedidos realizados por los usuarios. Modela los pedidos y su relación con los productos seleccionados, calcula el precio total y permite gestionar el ciclo de vida del pedido. Constituye el núcleo funcional del flujo de compra dentro de la aplicación.

3 Aplicación `api_view_roles` (Roles)

Esta miniaplicación implementa la gestión de roles del sistema mediante *Django REST Framework*. Su finalidad es mantener el catálogo de roles disponibles (por ejemplo, *Administrador*, *Propietario*, *Cliente*) y exponer operaciones CRUD sobre dicha entidad a través de un `ModelViewSet`.

3.1 Modelo `Role`

El modelo `Role` define la tabla de roles con los siguientes campos:

- `id_role`: clave primaria autoincremental.
- `name`: nombre del rol (único).
- `description`: descripción textual del rol.

Además, el modelo define un orden por defecto (`ordering`) basado en `id_role`.

3.2 Serialización

La serialización se realiza mediante `RoleSerializer`, que expone los campos: `id_role`, `name` y `description`. El campo `id_role` es de solo lectura.

3.3 Vista y rutas

La funcionalidad se concentra en `RoleViewSet` (un `ModelViewSet`) y se publica mediante un `DefaultRouter`. La ruta base se monta bajo el prefijo `/role/` en el enrutado principal del proyecto.

3.4 Autenticación y permisos

- **Autenticación**: se utiliza `CookieJWTAuthentication`, que intenta recuperar el token JWT desde la cookie `access`.
- **Permisos**: se aplican dos permisos:
 - `HasJWTUser`: permite el acceso siempre que `request.user` no sea `None`.
 - `CanUseRoles`: restringe las acciones `list`, `create`, `update`, `partial_update` y `destroy` a usuarios con rol *Administrador*.

Nota sobre el comportamiento actual: en la implementación presente, la acción `retrieve` (obtener un rol concreto) no está incluida en las acciones restringidas por `CanUseRoles`. Por tanto, el endpoint de detalle puede quedar accesible incluso sin rol de administrador (y potencialmente sin JWT, dependiendo de cómo se resuelva `request.user` en ausencia de cookie).

3.5 Operaciones disponibles y respuestas

Al ser un `ModelViewSet`, la aplicación expone las operaciones estándar:

- **Listar roles** (GET `/role/`)
 - **200 OK**: devuelve una lista JSON de roles.
 - **403 Forbidden**: si el usuario no es *Administrador*.
- **Obtener un rol** (GET `/role/{id_role}/`)
 - **200 OK**: devuelve el rol solicitado.

- **404 Not Found:** si no existe un rol con ese identificador.
- **Crear un rol (POST /role/)**
 - **201 Created:** devuelve el rol creado.
 - **400 Bad Request:** si faltan campos o hay violaciones de validación (p. ej., `name` duplicado).
 - **403 Forbidden:** si el usuario no es *Administrador*.
- **Actualizar un rol completo (PUT /role/{id_role}/)**
 - **200 OK:** devuelve el rol actualizado.
 - **400 Bad Request:** si los datos no son válidos.
 - **403 Forbidden:** si el usuario no es *Administrador*.
 - **404 Not Found:** si el rol no existe.
- **Actualizar parcialmente un rol (PATCH /role/{id_role}/)**
 - **200 OK:** devuelve el rol actualizado parcialmente.
 - **400 Bad Request:** si los datos no son válidos.
 - **403 Forbidden:** si el usuario no es *Administrador*.
 - **404 Not Found:** si el rol no existe.
- **Eliminar un rol (DELETE /role/{id_role}/)**
 - **204 No Content:** eliminación correcta.
 - **403 Forbidden:** si el usuario no es *Administrador*.
 - **404 Not Found:** si el rol no existe.

4 Aplicación `viewset_usuarios` (Usuarios)

La miniaplicación `viewset_usuarios` implementa la gestión de usuarios del sistema mediante *Django REST Framework*. Su núcleo es un `ModelViewSet` que expone operaciones CRUD sobre el modelo `User`. Esta aplicación centraliza la información de cuenta (credenciales y datos personales) y asocia cada usuario a un rol (`Role`), lo que permite aplicar control de acceso en otros módulos (establecimientos, productos, etc.).

4.1 Modelo `User`

El modelo `User` define la entidad usuario con los siguientes campos:

- `id_user`: clave primaria autoincremental.
- `username`: nombre de usuario (único).

- `email`: correo electrónico (único).
- `first_name`, `last_name`: datos personales.
- `password`: contraseña (almacenada como texto en este proyecto).
- `birth_date`: fecha de nacimiento (opcional).
- `date_joined`: fecha/hora de alta (autogenerada).
- `points`: puntos del usuario (por defecto 0).
- `role`: clave foránea a `Role` (obligatoria).

El modelo define un orden por defecto (`ordering`) por `id_user`.

4.2 Serialización

La serialización se realiza mediante `UserSerializer`, que expone: `id_user`, `username`, `email`, `first_name`, `last_name`, `password`, `birth_date`, `date_joined`, `points` y `role`.

Campos de solo lectura:

- `id_user`
- `date_joined`
- `points`

4.3 Vista, rutas y autenticación

La funcionalidad se concentra en `UserViewSet` (un `ModelViewSet`) y se publica mediante un `DefaultRouter`. La ruta base se monta bajo el prefijo `/user/`.

- **Autenticación:** `CookieJWTAuthentication`, que recupera el JWT desde la cookie `access` y resuelve el usuario a partir del `user_id` contenido en el token.
- **Permisos:**
 - `HasJWTUser`: permite el acceso si `request.user` no es `None`.
 - `CanListUser`: restringe la acción `list` a usuarios con rol *Administrador*.

Observación relevante: `HasJWTUser` únicamente comprueba que `request.user` no sea `None`. En *DRF* es habitual que, si no se autentica correctamente, `request.user` sea un `AnonymousUser` (que no es `None`). En consecuencia, el control de acceso efectivo puede no bloquear peticiones sin cookie JWT, salvo que otros componentes del framework estén endureciendo ese comportamiento en tiempo de ejecución.

4.4 Operaciones disponibles y respuestas

Al ser un `ModelViewSet`, se exponen las operaciones estándar:

- **Listar usuarios** (GET `/user/`)
 - **200 OK**: devuelve una lista JSON con los usuarios.
 - **403 Forbidden**: si el usuario no tiene rol *Administrador* (controlado por `CanListUser`).
- **Obtener un usuario** (GET `/user/{id_user}/`)
 - **200 OK**: devuelve el usuario solicitado.
 - **404 Not Found**: si no existe un usuario con ese identificador.
- **Crear un usuario** (POST `/user/`)
 - **201 Created**: devuelve el usuario creado.
 - **400 Bad Request**: si faltan campos obligatorios o hay validaciones incumplidas (por ejemplo, `username` o `email` duplicados, o `role` inexistente/no válido).
- **Actualizar un usuario completo** (PUT `/user/{id_user}/`)
 - **200 OK**: devuelve el usuario actualizado.
 - **400 Bad Request**: si los datos no son válidos o faltan campos requeridos para una actualización completa.
 - **404 Not Found**: si el usuario no existe.
- **Actualizar parcialmente un usuario** (PATCH `/user/{id_user}/`)
 - **200 OK**: devuelve el usuario tras aplicar la modificación parcial.
 - **400 Bad Request**: si los datos no son válidos.
 - **404 Not Found**: si el usuario no existe.
- **Eliminar un usuario** (DELETE `/user/{id_user}/`)
 - **204 No Content**: eliminación correcta.
 - **404 Not Found**: si el usuario no existe.

4.5 Tipos de respuesta (forma del payload)

De manera general, las respuestas de esta aplicación siguen el patrón típico de DRF:

- En **200/201**, el cuerpo suele ser un JSON con la representación serializada del usuario (o lista de usuarios).
- En **204**, la respuesta no contiene cuerpo.

- En **400**, el cuerpo suele incluir un JSON con errores por campo (por ejemplo, `{"email": ["This field must be unique."]}`).
- En **403**, el cuerpo suele indicar falta de permisos (mensaje estándar de DRF).
- En **404**, el cuerpo indica que el recurso no existe.

5 Aplicación `api_view_login` (Login y autenticación)

La miniaplicación `api_view_login` concentra la funcionalidad de **inicio de sesión** y la definición del mecanismo de **autenticación** utilizado por el resto de endpoints del sistema. Su objetivo es validar credenciales (`username` y `password`), generar tokens JWT mediante *SimpleJWT* y entregar el token de acceso tanto en el cuerpo de la respuesta como en una cookie HTTP-only.

5.1 Rutas expuestas

La aplicación define un único endpoint:

- **Login:** POST `/auth/login/`

Esta ruta se declara en `api_view_login/urls.py` como: `path("login/", login_view, name="login")`.

5.2 Función `login_view`

La función `login_view`:

- Acepta exclusivamente peticiones POST (decorador `require_http_methods(["POST"])`).
- Desactiva CSRF (`csrf_exempt`), al tratarse de un endpoint orientado a API.
- Lee el cuerpo de la petición como JSON y valida la presencia de `username` y `password`.
- Busca el usuario en base de datos mediante `User.objects.get(username=...)`.
- Comprueba la contraseña mediante comparación directa (`str(user.password) == str(password)`).
- Genera un token `refresh` y un token `access` asociados al usuario con `RefreshToken.for_user(user)`.
- Inserta **claims personalizados** en el token:
 - `user_id` (tomado de `user.id_user`)
 - `username`
- Devuelve la respuesta con `access`, `refresh` y `user_id`, y adicionalmente fija una cookie llamada `access`.

5.3 Cookie de autenticación

El token de acceso se guarda en la cookie `access` mediante `response.set_cookie(...)` con los siguientes parámetros:

- `httponly=True`: el token no es accesible desde JavaScript del cliente.
- `secure=False`: en la configuración actual no se exige HTTPS.
- `samesite="Strict"`: restringe el envío de la cookie en contextos cross-site.

5.4 Clase `CookieJWTAuthentication`

En `api_view_login/authentication.py` se define `CookieJWTAuthentication`, que extiende `JWTAuthentication` y autentica leyendo el token desde la cookie `access`:

- Si la cookie no existe, devuelve `None` (no autentica).
- Si el token es inválido, lanza `AuthenticationFailed("Invalid token")`.
- Si el token es válido, extrae `user_id` del token y recupera el usuario con `User.objects.get(id_user=...)`.

5.5 Respuestas posibles del endpoint `POST /auth/login/`

- **200 OK** (login correcto)
Devuelve un JSON con:
 - `access`: token JWT de acceso.
 - `refresh`: token JWT de refresco.
 - `user_id`: identificador del usuario.Además, fija la cookie HTTP-only `access` con el token de acceso.
- **400 Bad Request** (cuerpo inválido)
Si el cuerpo no es JSON parseable:
 - `{"error": "Invalid body"}`
- **400 Bad Request** (campos ausentes)
Si faltan `username` o `password`:
 - `{"error": "Missing fields"}`
- **401 Unauthorized** (usuario inexistente)
Si no existe el usuario:
 - `{"error": "Invalid username"}`
- **401 Unauthorized** (contraseña incorrecta)
Si la contraseña no coincide:
 - `{"error": "Invalid password"}`

6 Aplicación `api_view_establecimiento` (Establecimientos)

La miniaplicación `api_view_establecimiento` implementa la gestión de los establecimientos registrados en la plataforma. Su objetivo es modelar los comercios disponibles, asociarlos a un usuario propietario y exponer operaciones CRUD mediante endpoints basados en *API Views*. A diferencia de otras aplicaciones, no utiliza `ModelViewSet`, sino vistas funcionales específicas para cada operación.

6.1 Modelo Establecimiento

El modelo `Establecimiento` define la entidad comercio con los siguientes campos:

- `id_establecimiento`: clave primaria autoincremental.
- `nombre`: nombre del establecimiento.
- `direccion`: dirección física.
- `telefono`: número de contacto.
- `email`: correo electrónico del establecimiento.
- `id_propietario`: clave foránea a `User`, que identifica al propietario del comercio.

6.2 Rutas expuestas

La aplicación define las siguientes rutas bajo el prefijo `/establecimiento/`:

- GET `/establecimiento/list/`
- POST `/establecimiento/create/`
- GET `/establecimiento/details/{id}/`
- PUT `/establecimiento/update/{id}/`
- DELETE `/establecimiento/delete/{id}/`

6.3 Autenticación y permisos

Todas las vistas utilizan:

- **Autenticación**: `CookieJWTAuthentication`.
- **Permiso base**: `HasJWTUser`.

Adicionalmente, se aplican comprobaciones manuales de rol mediante funciones auxiliares:

- Usuarios con rol *Administrador* pueden gestionar cualquier establecimiento.
- Usuarios con rol *Propietario* solo pueden crear, modificar o eliminar establecimientos de los que son propietarios.

6.4 Análisis de las funciones

6.4.1 list_establecimientos

Ruta: GET /establecimiento/list/

Devuelve el listado completo de establecimientos registrados.

- **200 OK:** devuelve una lista JSON con todos los establecimientos.
- **403 Forbidden:** si el usuario no tiene permisos suficientes.

6.4.2 create_establecimiento

Ruta: POST /establecimiento/create/

Crea un nuevo establecimiento a partir de los datos enviados en el cuerpo de la petición.

- **201 Created:** establecimiento creado correctamente.
- **400 Bad Request:** datos incompletos o inválidos.
- **403 Forbidden:** si el usuario no tiene rol de *Administrador* ni *Propietario*.

6.4.3 detail_establecimiento

Ruta: GET /establecimiento/details/{id}/

Devuelve la información de un establecimiento concreto identificado por su ID.

- **200 OK:** devuelve el establecimiento solicitado.
- **404 Not Found:** si el establecimiento no existe.

6.4.4 update_establecimiento

Ruta: PUT /establecimiento/update/{id}/

Actualiza los datos de un establecimiento existente.

- **200 OK:** establecimiento actualizado correctamente.
- **400 Bad Request:** datos inválidos.
- **403 Forbidden:** si el usuario no es administrador ni propietario del establecimiento.
- **404 Not Found:** si el establecimiento no existe.

6.4.5 delete_establecimiento

Ruta: DELETE /establecimiento/delete/{id}/

Elimina un establecimiento del sistema.

- **204 No Content:** eliminación correcta.
- **403 Forbidden:** si el usuario no es administrador ni propietario del establecimiento.
- **404 Not Found:** si el establecimiento no existe.

6.5 Tipos de respuesta

Las respuestas siguen un esquema homogéneo:

- **200 / 201:** JSON con la representación del establecimiento o lista de establecimientos.
- **204:** respuesta sin cuerpo.
- **400:** JSON con mensajes de error asociados a validación de datos.
- **403:** error de permisos insuficientes.
- **404:** recurso no encontrado.

7 Aplicación api_view_pedidos (Pedidos)

La miniaplicación `api_view_pedidos` implementa la lógica asociada a la creación y gestión de pedidos dentro de la plataforma. Su función principal es permitir que los usuarios realicen pedidos seleccionando productos, calcular el precio total y gestionar el ciclo de vida del pedido. Esta aplicación constituye el núcleo funcional del flujo de compra del sistema.

7.1 Modelo Pedido

El modelo `Pedido` define la entidad pedido con los siguientes campos:

- `id_pedido`: clave primaria autoincremental.
- `fecha_pedido`: fecha y hora de creación del pedido.
- `precio_total`: importe total del pedido.
- `usuario`: clave foránea a `User`, que identifica al usuario que realiza el pedido.
- `productos`: relación `ManyToMany` con `Producto`, que representa los productos incluidos en el pedido.

El modelo incluye un método auxiliar `to_dict()` que devuelve una representación serializada del pedido, incluyendo la lista de productos asociados.

7.2 Rutas expuestas

La aplicación define las siguientes rutas bajo el prefijo `/pedido/`:

- **POST** `/pedido/create/`
- **GET** `/pedido/list/`
- **GET** `/pedido/detail/{id}/`
- **PUT** `/pedido/update/{id}/`
- **DELETE** `/pedido/delete/{id}/`

7.3 Autenticación y permisos

Todas las vistas utilizan:

- **Autenticación:** `CookieJWTAuthentication`.
- **Permiso base:** `HasJWTUser`.

Adicionalmente, se realizan comprobaciones de rol:

- Usuarios con rol *Administrador* pueden acceder y gestionar cualquier pedido.
- Usuarios con rol *Cliente* solo pueden crear pedidos y consultar o modificar los suyos propios.

7.4 Análisis de las funciones

7.4.1 `create_pedido`

Ruta: **POST** `/pedido/create/`

Crea un nuevo pedido a partir de un conjunto de identificadores de productos enviados en el cuerpo de la petición. La función calcula el precio total sumando los precios de los productos seleccionados y asocia el pedido al usuario autenticado.

- **201 Created:** pedido creado correctamente.
- **400 Bad Request:** productos inexistentes o datos inválidos.

7.4.2 `list_pedidos`

Ruta: **GET** `/pedido/list/`

Devuelve el listado de pedidos. El contenido del listado depende del rol del usuario autenticado.

- **200 OK:** lista de pedidos en formato JSON.
- **403 Forbidden:** acceso denegado por permisos insuficientes.

7.4.3 detail_pedido

Ruta: GET /pedido/detail/{id}/

Devuelve la información detallada de un pedido concreto, incluyendo los productos asociados.

- **200 OK:** pedido encontrado y devuelto correctamente.
- **404 Not Found:** el pedido no existe.

7.4.4 update_pedido

Ruta: PUT /pedido/update/{id}/

Permite modificar un pedido existente, normalmente alterando el conjunto de productos asociados y recalculando el precio total.

- **200 OK:** pedido actualizado correctamente.
- **400 Bad Request:** datos inválidos.
- **403 Forbidden:** el usuario no tiene permisos para modificar el pedido.
- **404 Not Found:** el pedido no existe.

7.4.5 delete_pedido

Ruta: DELETE /pedido/delete/{id}/

Elimina un pedido del sistema.

- **204 No Content:** eliminación correcta.
- **403 Forbidden:** permisos insuficientes.
- **404 Not Found:** el pedido no existe.

7.5 Tipos de respuesta

Las respuestas de esta aplicación siguen un esquema homogéneo:

- **200 / 201:** JSON con la representación del pedido o lista de pedidos.
- **204:** respuesta sin cuerpo.
- **400:** JSON con errores de validación o referencias inválidas.
- **403:** error de autorización.
- **404:** recurso no encontrado.

8 Aplicación `api_view_pedidos` (Pedidos)

La miniaplicación `api_view_pedidos` implementa la gestión completa de los pedidos realizados por los usuarios dentro de la plataforma. Su objetivo es coordinar la relación entre usuarios y productos, permitiendo crear pedidos, calcular su importe total y consultar o modificar su estado. Esta aplicación representa el núcleo del flujo funcional de la aplicación de reparto.

8.1 Modelo Pedido

El modelo `Pedido` define la entidad pedido con los siguientes campos:

- `id_pedido`: clave primaria autoincremental.
- `fecha_pedido`: fecha y hora de creación del pedido.
- `precio_total`: precio total calculado del pedido.
- `usuario`: clave foránea a `User`, que identifica al usuario que realiza el pedido.
- `productos`: relación `ManyToMany` con `Producto`, que representa los productos incluidos en el pedido.

El modelo incluye un método auxiliar `to_dict()`, utilizado para generar una representación serializada del pedido junto con la información de los productos asociados.

8.2 Rutas expuestas

La aplicación define las siguientes rutas bajo el prefijo `/pedido/`:

- `POST /pedido/create/`
- `GET /pedido/list/`
- `GET /pedido/detail/{id}/`
- `PUT /pedido/update/{id}/`
- `DELETE /pedido/delete/{id}/`

8.3 Autenticación y control de acceso

Todas las vistas de esta aplicación utilizan:

- **Autenticación**: `CookieJWTAuthentication`, basada en un token JWT almacenado en la cookie `access`.
- **Permiso base**: `HasJWTUser`, que exige que el usuario esté autenticado.

Además, se realizan comprobaciones adicionales de rol:

- Los usuarios con rol *Administrador* pueden consultar, modificar y eliminar cualquier pedido.
- Los usuarios con rol *Cliente* solo pueden crear pedidos y acceder a aquellos que les pertenecen.

8.4 Análisis de las funciones

8.4.1 create_pedido

Ruta: POST /pedido/create/

Crea un nuevo pedido a partir de una lista de identificadores de productos enviada en el cuerpo de la petición. La función calcula automáticamente el precio total sumando los precios de los productos seleccionados y asocia el pedido al usuario autenticado.

- **201 Created:** pedido creado correctamente.
- **400 Bad Request:** datos inválidos o productos inexistentes.

8.4.2 list_pedidos

Ruta: GET /pedido/list/

Devuelve un listado de pedidos. El contenido de la respuesta depende del rol del usuario autenticado, mostrando todos los pedidos o únicamente los propios.

- **200 OK:** lista de pedidos en formato JSON.
- **403 Forbidden:** permisos insuficientes.

8.4.3 detail_pedido

Ruta: GET /pedido/detail/{id}/

Devuelve la información detallada de un pedido concreto, incluyendo los productos asociados.

- **200 OK:** pedido encontrado.
- **404 Not Found:** el pedido no existe.

8.4.4 update_pedido

Ruta: PUT /pedido/update/{id}/

Permite modificar un pedido existente, normalmente actualizando los productos asociados y recalculando el precio total.

- **200 OK:** pedido actualizado correctamente.

- **400 Bad Request:** datos inválidos.
- **403 Forbidden:** el usuario no tiene permisos para modificar el pedido.
- **404 Not Found:** el pedido no existe.

8.4.5 delete_pedido

Ruta: DELETE /pedido/delete/{id}/
Elimina un pedido del sistema.

- **204 No Content:** eliminación correcta.
- **403 Forbidden:** permisos insuficientes.
- **404 Not Found:** el pedido no existe.

8.5 Tipos de respuesta

Las respuestas generadas por esta aplicación siguen un patrón uniforme:

- **200 / 201:** JSON con la representación del pedido o un listado de pedidos.
- **204:** respuesta sin contenido.
- **400:** JSON con errores de validación.
- **403:** error de autorización.
- **404:** recurso no encontrado.

9 Cómo ejecutar la aplicación

La puesta en marcha consiste en: clonar el repositorio, instalar dependencias y ejecutar las migraciones de Django antes de arrancar el servidor.

Requisitos

- Tener instalado **Python 3** y **pip**.
- (Recomendado) Usar un entorno virtual.

9.1 1) Clonar el repositorio

Clonado del proyecto

```
1 git clone https://github.com/Alvaro2406/master-webdev-Pr-ctica-4
2 cd master-webdev-Pr-ctica-4
```

9.2 2) Crear (opcional) y activar entorno virtual

Entorno virtual (recomendado)

```
1 python -m venv .venv
2
3 # Linux/macOS:
4 source .venv/bin/activate
5
6 # Windows (PowerShell):
7 # .venv\Scripts\Activate.ps1
```

9.3 3) Instalar dependencias

Instalación desde requirements.txt

```
1 pip install -r requirements.txt
```

9.4 4) Migraciones y arranque del servidor

Migraciones y ejecución

```
1 python manage.py makemigrations
2 python manage.py migrate
3 python manage.py runserver
```

Tras ejecutar `python manage.py runserver`, la aplicación quedará accesible en la URL local que muestre la consola (por defecto, `http://127.0.0.1:8000/`).